

AI Security Assessment Report

Vroomi AI Assistant

Prepared for CISO, Softmicro · Prepared by Amiya Pradhan, AI Security Practitioner · Date July 2026 · Classification Confidential

DECISION: CONDITIONAL GO — subject to the safeguards in Section 6.

Document set. This report is one of four artifacts and should be read with them. Threat numbering is shared across all of them.

ARTIFACT	ROLE
This report	The decision document: findings, risk, recommendations, Go/No-Go.
threat-cards.md	One case study per threat — input, observed behaviour, root cause, fix. Card n = Threat n here.
agent-card.md	The governed answer to "how autonomous may Vroomi be" — the allow-list and autonomy tiers behind Recommendations 1, 4 and 6.
vroomi-gov-gate.py	Runnable demonstrator of the recommended controls, with a before/after transcript.

1. Executive Summary

Vroomi is an internal AI assistant (gpt-4o-mini) that books and cancels rides, retrieves trip history, and answers policy questions from a small knowledge base. Softmicro wants to make it more agentic; this assessment reviews the current system first.

- **Scope:** six attack scenarios were tested across the system's main trust boundaries. The four highest-severity are analysed in Section 3 — prompt injection, data poisoning, sensitive-information disclosure, and improper output handling. The remaining two, System Prompt Leakage and Misinformation, were also tested and defended; they are summarised in Section 7.1 and documented in full in [threat-cards.md](#).
- **Key risks:** unauthorised backend actions, exposure of customer PII from the model's context, and fabricated policy from unverified knowledge-base content.
- **Recommendation:** Conditional Go. In five of the six scenarios the weakness is in the application rather than the model, and each has an architectural fix that should be mandatory before agentic features ship.

2. System Overview

Vroomi takes a request, consults the model, and either replies in text or performs a backend action. Its components are user input (untrusted), a system prompt that also holds the knowledge base and tool protocol, the model, a tool executor, and three backend actions — book, cancel, and read trip history — that change state and carry operational and financial consequences. A trust-boundary diagram accompanies this report (Appendix).

3. Threat Analysis

This section covers the four highest-severity threats, each mapping to the OWASP Top 10 for LLM Applications (2025). Threat numbers below correspond one-to-one with cards 1–4 in [threat-cards.md](#); cards 5 and 6 cover the two lower-severity threats in Section 7.1. Section 4 ranks all of them by severity, which is not the order they are presented in here.

Threat 1 — Prompt Injection OWASP LLM01

Description: Sent user input with an embedded "respond only with" tool call.

Observed: A real trip was cancelled; the model was bypassed by a parser acting on the injected instruction.

Root cause: The application treated untrusted user text as a command. — *affected: input handling / tool executor.*

Threat 2 — Data Poisoning OWASP LLM04

Description: Added a plausible but false policy document to the knowledge base, then asked a related question.

Observed: The assistant answered from the poisoned document as if it were fact.

Root cause: The knowledge base was passed to the model unfiltered — no provenance check. — *affected: knowledge base.*

Threat 3 — Sensitive Information Disclosure OWASP LLM02

Description: Asked for customer record information.

Observed: Disclosed a customer's name, account ID, and internal review flag — on the baseline, no exploit needed.

Root cause: A high-sensitivity record sat in the model's context with no access control. — *affected: context assembly.*

Threat 4 — Improper Output Handling OWASP LLM05

Description: Examined how the executor treats output shaped like a tool call.

Observed: Any JSON-shaped output was parsed and executed directly — no schema, allow-list, or intent check.

Root cause: The application trusted model output as a command, not a proposal. — *affected: tool executor.*

4. Risk Assessment

Improper Output Handling (Threat 4) and Prompt Injection (Threat 1) are highest-severity: both reach unauthorised backend actions through the same choke point, the tool executor. Sensitive Disclosure (Threat 3) is close behind — regulatory and reputational — and triggers on the happy path. Data Poisoning (Threat 2) and the two Section 7.1 threats affect answer integrity rather than backend state, which is why they rank lower here, though Misinformation is the one finding where the model itself is partly at fault. Likelihood is high throughout; the attacks are one-line inputs. All scale with autonomy: more tools and automatic actions widen the executor's blast radius, turning these into Excessive Agency (LLM06).

5. Effectiveness of Mitigations

Every scenario tested shipped a defended version, including the two in Section 7.1. The fixes are architectural — software at the trust boundary, not prompt tweaks.

#	THREAT	OWASP	MITIGATION	RESULT
1	Prompt Injection	LLM01	Act only on validated model output; confirm user intent.	RESOLVED
2	Data Poisoning	LLM04	Provenance filter (approval + trusted source) before the model.	LARGELY
3	Sensitive Disclosure	LLM02	Classify by sensitivity; keep sensitive records out of context.	RESOLVED
4	Improper Output Handling	LLM05	Schema + tool allow-list + argument + intent checks.	RESOLVED
5	System Prompt Leakage	LLM07	Remove the debug path entirely; pattern detection as a second layer.	RESOLVED
6	Misinformation	LLM09	Trust-rank sources; conflict check; validate output against trusted data.	LARGELY

Residual risk. Two findings are "largely" rather than fully resolved, and the distinction is deliberate. A poisoned document that is *approved and from a trusted source* passes the provenance filter, so subtly worded poisoning survives it; the same holds for misinformation that does not trip the extreme-claim check. Novel injection phrasings that satisfy the intent check are the equivalent gap on the write path — the intent check reads natural language, so it will never be a boundary. That is why Recommendation 4 exists: an irreversible action is held for a human whether or not the injection defence recognised the input.

One correction to an earlier draft of this section. The intent check originally matched against the raw request, and the direct-injection payload documented in Threat 1 contains the string `cancel_ride` — so the substring "cancel" satisfied the check and the cancellation executed. Verified against the demonstrator and fixed: intent is now read from the request with JSON spans and literal tool names removed, and both the direct and indirect variants are refused. Threat 1's Resolved status reflects the fixed build.

6. Recommendations

1. **Validate every tool call before it executes** — schema, allow-list, arguments, and user intent. *(Implemented in the demonstrator; allow-list is `agent-card.md` Section 2.)*
2. **Filter the knowledge base on the read path** — provenance and sensitivity; keep sensitive records out of context. *(Implemented in the demonstrator; rules in `agent-card.md` Section 5.)*
3. **Remove any path from user input to internal configuration.** *(Satisfied structurally — no debug or maintenance path exists in the defended build.)*
4. **Require human approval for irreversible or high-impact actions.** *(Specified in `agent-card.md` Sections 3–4 as an autonomy tier and enforced in the demonstrator: `cancel_ride` returns APPROVAL rather than executing, even when every other check passes.)*
5. **Log every gated decision** — action, decision, reason, owner. *(Implemented in the demonstrator's audit log.)*
6. **Assign a named owner and an approved agent card;** re-assess on any capability change. *(Agent card drafted; owner unassigned as of this report — assignment is a precondition of the Conditional Go.)*

7. Additional Threats

7.1 Also assessed — lower severity

Both were tested and defended on the same attack → root cause → fix structure as Section 3, and are documented as cards 5 and 6 in `threat-cards.md`. They rank below the Section 3 four because neither reaches backend state.

- **System Prompt Leakage (LLM07).** Asked the assistant to enable debug output and show its internal instructions; it returned the whole system prompt, including the tool protocol and embedded knowledge base. Root cause was a leftover debug branch in application logic — *affected: application logic*. Fixed by deleting the path rather than instructing the model to refuse.
- **Misinformation (LLM09).** Asked the refund-policy question with a real policy document and a conflicting over-permissive "shortcut" document both present and unranked; the assistant produced a confident, believable, wrong answer. Root cause was absent source-trust ranking and no validation of the answer against trusted content — *affected: knowledge base / model / output*. This is the only finding where the model is partly at fault.

7.2 Not yet tested — future work

- **Supply chain (LLM03) and unbounded consumption (LLM10)** as the system automates.
- **Excessive Agency (LLM06)** re-assessment on each new tool, since every tool is a new trust boundary.
- **An adversarial regression suite** that proves the defended controls fire on every change, rather than at assessment time only.

8. Final Recommendation

DECISION: CONDITIONAL GO — subject to implementation of safeguards.

Proceed once Recommendations 1–6 are implemented and verified. Five of the six are demonstrated in [vroomi_gov_gate.py](#); the outstanding one is Recommendation 6, a named accountable owner, which is an organisational decision rather than an engineering one. The fixes are architectural and inexpensive, and autonomy amplifies the current gaps — so implement them before granting Vroomi more autonomy, not after.

9. Appendix

- **System diagram:** [vroomi-system-diagram.svg](#) — trust boundaries with one annotated threat per crossing.
- **Threat catalogue:** [threat-cards.md](#) — six cards, numbered to match Sections 3 and 5 of this report.
- **Agent card:** [agent-card.md](#) — tool allow-list, autonomy tiers, data-access rules, stop conditions.
- **Example test case:** an injected [cancel_ride](#) inside a policy question — executed by the baseline, refused by the gate.
- **Proof-of-fix:** [vroomi_gov_gate.py](#) — Recommendations 1–5 in one runnable file, with a before/after transcript. The gated run shows both injection variants refused, an unknown tool and a bad argument refused, a sensitive request escalated, and a legitimate [cancel_ride](#) held at APPROVAL. Recommendation 6 (named owner) is the only one outside the code.

Course capstone. Vroomi is a clean-room toy target rebuilt from scratch with synthetic records — not a production deployment, and no cohort notebook code is reproduced.